

Introduction

What Almanac is, what works today, and what is still on the roadmap.

Shravan Goswami / <https://shravangoswami.com>

Almanac is a publishing framework for technical and scientific writing. It ships as an Astro integration: you install it from npm, configure it in `astro.config.mjs`, and write Markdown in a top-level `docs/` directory. One source tree produces a documentation site and a blog, and eventually executed code output and print formats.

This site is built with Almanac, so what you are reading is the framework rendering its own documentation.

What works today

- **Docs routing:** every Markdown file under `docs/` becomes a page, with no per-file layout wiring and no route to add.
- **Sidebars:** group pages explicitly in config, or leave sidebar out and Almanac groups them by directory. See [Writing docs](#).
- **A previous/next pager** at the foot of each docs page, ordered by the sidebar.
- **A table of contents** built from each page's `##` and `###` headings, with scroll-spy highlighting of the section you're reading.
- **Instant search** powered by [Pagefind](#), on a header button and `Cmd+K`. No external service, no API key.
- **A blog** with tags, an RSS feed, hero images, and reading times. Off by default, turned on with `blog.enabled`.
- **42 built-in themes**, a light/dark toggle, and a theme picker. See [Theming](#).
- **A build-time OG image** rendered with [Satori](#), plus JSON-LD structured data and a generated `robots.txt`.
- **Copy-to-clipboard code blocks:** every fenced block gets a copy button.
- **An optional terminal-recording player**, `<PromptCast />`, for documenting CLIs.
- **Config validated with Zod**, so a typo fails the build with the path of the problem instead of rendering a broken page.

Beyond the web page

These are built and off by default, because each one is a bigger commitment than rendering Markdown:

- **Executable code blocks.** Run JavaScript, TypeScript, Python, and R during the build and embed the real output, cached on the code, the language, and the runtime version. Behind `future.execute`.
- **MyST parsing.** Cross references that write their own text, numbered figures, directives, and roles. Behind `future.myst`.
- **PDF output.** Every docs page typeset by Typst from the same tree the web page came from, executed output included, plus the whole thing as one book with a cover and a clickable table of contents. Styling is yours, down to a full Typst template. Behind `future.pdf`.
- **Versions and translations.** Both inherit unchanged pages instead of copying them, and build in a single pass.

What is still missing

- **Installing packages into a runtime.** Pyodide gets the Python standard library and WebR gets base R. Anything beyond that needs a [Jupyter kernel](#) with the packages already installed.
- **Rich output from the local runtimes.** A Jupyter kernel's images, SVG, HTML, and JSON are rendered; Pyodide and WebR emit text only.
- **Citations and bibliographies.** `{cite}` parses but there is no bibliography renderer.

Where the pieces live

Almanac is a package you depend on, not a repository you clone. Two things belong to you: `astro.config.mjs`, which holds the config, and the content directories (`docs/`, plus `blog/` if you enable it). Everything else, the routes, layouts, components, themes, and search wiring, comes from the package and updates when you update the dependency.

Next steps

- [Installation](#) covers both a new site and adding Almanac to an existing Astro project.
- [Quick start](#) is the short list of things to change to make the site yours.
- [Configuration reference](#) documents every option, its default, and what is not wired up yet.